

HPC

Arthur Lequertier

August 26, 2026

Contents

1	Introduction	1
2	Les grands principes de l'HPC	3
2.1	La taxonomie de Flynn	3
2.2	Décomposition du problème	4
2.3	La mémoire	4
2.3.1	La mémoire partagée	4
2.3.2	La mémoire distribuée	5
2.3.3	Les architectures hybrides	5
3	calcul parallèle sur GPU	6
3.1	Qu'est ce qu'un GPU ?	6
3.2	Comment évaluer un GPU	6
3.3	CUDA	7

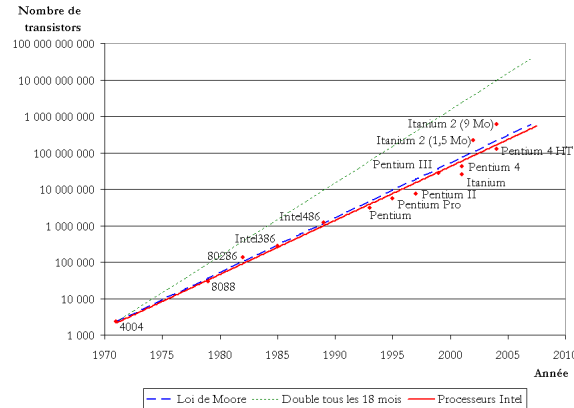
1 Introduction

Le contexte est simple. Les problèmes informatiques sont de plus en plus complexes, de par le nombre de données, la précision exigée, et les interactions à prendre en compte. Heureusement, la technologie de calcul scientifique suit le progrès et permet d'accompagner les nouveaux modèles informatiques ¹ Cependant, un simple processeur de calcul monocœur ne peut pas répondre à lui seul aux attentes des problématiques actuelles. Il est donc nécessaire de concevoir des architectures haute performance, répartissant les tâches entre plusieurs cœurs de calcul.

Pour rappel, la loi de Moore dit que le nombre de transistors intégrés sur une puce double environ tous les 18 à 24 mois, à coût comparable. La miniaturisation permet généralement d'améliorer les performances, la densité et l'efficacité

¹Ceci est un raccourci, le progrès sur le software et le hardware va de pair. L'exemple des réseaux de neurones, mis au placard pendant une vingtaine d'années, n'a trouvé son utilité que lorsque les clusters de calcul GPU ont atteint une puissance suffisante pour entraîner de grands modèles.

énergétique. Les circuits de des processeurs sont gravés de plus en plus finement, mais les technologies de fabrication les plus avancées ont atteint une limite, car annoncées autour de 2 nm ².



On se rapproche aujourd'hui de la taille d'atome de silicium (environ 0.2 nm) et ne peut évidemment pas continuer indéfiniment à réduire la taille des transistors. Au milieu des années 2000, la fin du Dennard scaling intervint. Jusque-là, réduire la taille des transistors permettait aussi de baisser leur tension, et donc de monter en fréquence, sans exploser la consommation et la dissipation thermique. Mais nous avons atteint une limite, car au-delà d'une certaine finesse de gravure, les courants de fuite deviennent trop importants. Donc, si je devais résumer pourquoi c'est un problème, je répondrais : "Tais-toi, c'est quantique !"

La solution a été d'utiliser ces transistors supplémentaires pour dupliquer des cœurs plutôt que d'accélérer un cœur unique, d'où l'essor du multicœur, et donc la nécessité de paralléliser les logiciels pour en tirer parti.

C'est le principe du calcul parallèle, mais ce n'est pas si simple que ça. Imaginons qu'un programme informatique tournant sur un CPU met 100 s pour tourner. Ce dernier peut être décomposé en un ensemble de tâches. Dans cet exemple, disons que 75 s représente des tâches parallèles (c.-à-d. pouvant s'exécuter simultanément), et 25 s correspondent à une partie qui doit obligatoirement être exécutée séquentiellement. Si on dispose de 10 processeurs, on peut théoriquement diviser les 75 secondes par 10. La partie parallélisable s'exécutera donc en 7.5 s, et la totalité du programme en $7.5 + 25 = 32.5$ s. Le programme est donc $\frac{100}{32.5} \approx 3$ fois plus rapide. La loi d'Amdahl s'écrit généralement

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

où :

²Ce sont des tailles caractéristiques utilisées pour différencier les générations de puces, ça ne signifie pas que chaque transistor possède une dimension de 2 nm

- $S(N)$ = accélération (speedup) obtenue avec N processeurs
- P = fraction du programme qui peut être parallélisée
- $1 - P$ = fraction qui doit rester séquentielle
- N = nombre de processeurs

La loi d'Amdahl permet de calculer la limite théorique du speedup lorsque le nombre de processeurs tend vers l'infini. Même une toute petite partie séquentielle peut fortement limiter les performances globales ³.

2 Les grands principes de l'HPC

2.1 La taxonomie de Flynn

Lorsque l'on parle de hardware informatique, il est possible d'utiliser la taxonomie de Flynn pour classer les ordinateurs et les processeurs selon la manière dont ils traitent les instructions et les données en parallèle.

L'idée repose sur deux questions :

1. Combien de flux d'instructions sont exécutés ?
2. Combien de flux de données sont traités ?

On obtient ainsi quatre grandes catégories :

	Un flux de données	Plusieurs flux de données
Une instruction	SISD	SIMD
Plusieurs instructions	MISD	MIMD

- SISD pour Single Instruction, Single Data : c'est le modèle informatique classique, voire élémentaire, où une seule instruction traite une seule donnée à la fois.
- SIMD pour Single Instruction, Multiple Data : ici, une seule instruction est appliquée simultanément à plusieurs données (pour des instructions vectorielles).
- MISD pour Multiple Instruction, Single Data : dans ce modèle, plusieurs instructions différentes travaillent sur un même flux de données.
- MIMD pour Multiple Instruction, Multiple Data : ici, plusieurs unités de calcul peuvent exécuter des instructions différentes sur des données différentes.

Les configurations de type MIMD correspondent le mieux à la plupart des architectures multicœurs modernes, car elles sont adaptées au calcul parallèle et au calcul distribué.

³La loi de Gustafson s'ajoute à la loi d'Amdahl, car en pratique, quand on a plus de ressources, on a tendance à augmenter la taille du problème plutôt qu'à garder la taille fixe.

2.2 Décomposition du problème

Lorsque l'on aborde un problème computationnel, deux grandes approches complémentaires s'offrent à nous.

- Décomposition de données : on découpe le domaine (ex. un maillage CFD) en sous-domaines traités en parallèle.
- Décomposition de tâches : on découpe le problème en tâches (threads) distinctes sur le plan fonctionnel.

2.3 La mémoire

La gestion de la mémoire est l'un des aspects les plus importants du calcul parallèle. En effet, disposer de nombreux cœurs de calcul ne suffit pas, car ces cœurs doivent aussi pouvoir accéder aux données dont ils ont besoin, et cet accès peut devenir le principal facteur limitant des performances. En effet, l'ordonnancement des tâches est important, car des résultats de threads peuvent être utiles à d'autres, les rendant limitants (on peut voir ça comme un diagramme de Gantt. On souhaite répartir le travail entre plusieurs unités de calcul).

Si on souhaite répartir le travail d'un programme entre plusieurs unités de calcul, une question essentielle se pose ! Où sont stockées les données ? Les cœurs de calcul partagent-ils la même mémoire ou bien chaque cœur possède-t-il sa propre mémoire ?

La réponse à cette question définit en grande partie le modèle de programmation parallèle. On distingue généralement trois grandes approches :

- la mémoire partagée (shared memory)
- la mémoire distribuée (distributed memory)
- les architectures hybrides

2.3.1 La mémoire partagée

Dans une architecture à mémoire partagée, plusieurs processeurs ou cœurs peuvent accéder à un espace mémoire commun. Ainsi, tous les processeurs peuvent potentiellement accéder aux mêmes variables. Le principal avantage est la simplicité de programmation : les threads peuvent partager directement des structures de données.

OpenMP est une API permettant de programmer des architectures à mémoire partagée. Le principe est de créer plusieurs threads qui travaillent en parallèle. Par exemple, si on travaille sur une boucle, il est possible de répartir cette dernière entre plusieurs threads.

```
1 #pragma omp parallel for
2 for (int i = 0; i < N; i++) {
3     C[i] = A[i] + B[i];
```

C'est très pratique pour les problèmes où les itérations sont relativement indépendantes.

La mémoire partagée pose cependant une difficulté majeure, car plusieurs threads peuvent modifier les mêmes données simultanément. C'est ce qu'on appelle une **condition de course** (race condition). Il faut donc utiliser des mécanismes de synchronisation (i.e., mutex, locks, barrières, etc...).

Une autre limite de la mémoire partagée est que la mémoire et les communications deviennent un goulot d'étranglement lorsque le nombre de processeurs augmente (problème de bande passante et de cohérence des caches). C'est l'une des raisons pour lesquelles les supercalculateurs recourent souvent à un modèle de mémoire distribuée.

2.3.2 La mémoire distribuée

Dans une architecture à mémoire distribuée, chaque processeur ou chaque nœud de calcul possède sa propre mémoire locale. Chaque processeur doit alors échanger explicitement des données avec les autres.

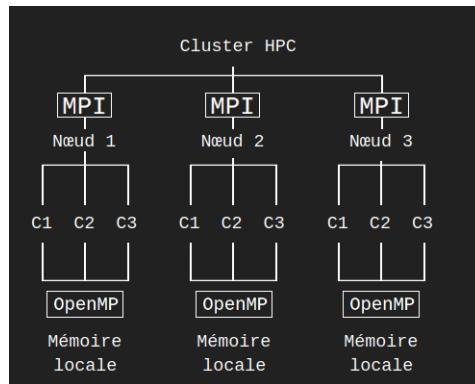
MPI (Message Passing Interface) est un standard qui permet à plusieurs processus de communiquer en échangeant des messages. Son principal avantage est la **scalabilité**. C'est-à-dire que si on veut augmenter la puissance de calcul, on peut ajouter des nœuds. Car chaque nouveau nœud apporte :

- Des processeurs supplémentaires
- De la mémoire supplémentaire
- Une capacité de calcul supplémentaire

2.3.3 Les architectures hybrides

Les architectures modernes combinent souvent les deux approches. Un supercalculateur peut être constitué de nombreux nœuds, chacun possédant plusieurs cœurs. On utilise alors :

- MPI entre les nœuds
- OpenMP à l'intérieur de chaque nœud



Par exemple, supposons que l'on souhaite traiter un ensemble de données énorme. MPI distribue les données entre les nœuds et OpenMP répartit ensuite le travail entre les cœurs de chaque nœud. On obtient ainsi deux niveaux de parallélisme.

Cependant, dans les architectures parallèles modernes, le coût des déplacements de données n'est pas à négliger par rapport au coût de calcul pur, bien au contraire. On peut avoir un processeur extrêmement puissant, mais si celui-ci doit attendre les données provenant de la mémoire ou d'un autre nœud, ses unités de calcul restent inactives ; c'est la hiérarchie mémoire (registre → cache L1 → cache L2 → cache L3 → RAM → mémoire d'un autre nœud sur le réseau). La performance d'une application parallèle dépend donc fortement de la manière dont les données circulent au sein de cette hiérarchie.

3 calcul parallèle sur GPU

3.1 Qu'est ce qu'un GPU ?

Pour Graphics Processing Unit, il s'agit d'unités de calcul dédiées au calcul, initialement conçues pour le rendu graphique. Alors qu'un CPU moderne possède généralement un nombre relativement limité de cœurs très puissants, un GPU possède énormément d'unités de calcul. Ainsi, un CPU fera rapidement des tâches complexes et variées, et un GPU sera spécialisé dans la réalisation de la même opération en parallèle énormément de fois.

3.2 Comment évaluer un GPU

Un GPU présente plusieurs caractéristique dépendant de sa conception.

- La fréquence : correspond à la vitesse à laquelle son processeur effectue des calculs, mesurée en mégahertz (MHz).
- Nombre de cœurs de calcul : La puce d'une GPU est constituée d'un certain nombre de cœurs de calcul (cœurs CUDA chez NVIDIA, et cœurs

de flux chez AMD). Chacun de ces coeurs est une unité de calcul exploitable pour paralléliser les calculs. Cependant, les coeurs de calcul de marques différentes et même de GPU de génération différente ne sont pas forcément comparables, ce qui fait de ce nombre un indicateur approximatif de la performance d'un GPU avant même de l'utiliser.

- Le TFLOPS : Cette unité mesure la puissance de calcul d'un processeur ou d'une carte graphique. T pour tera (10^{12}), FLOP pour *float operation*, et le S pour s^{-1} , il correspond à mille milliards d'opérations en virgule flottante par seconde, c'est-à-dire des calculs mathématiques avec des nombres décimaux. Plus ce nombre est élevé, plus l'appareil est rapide pour gérer des calculs.
- La VRAM : Il s'agit de la mémoire vive dédiée à la carte graphique. Bien que n'intervenant pas dans la vitesse des calculs, une VRAM trop petite peut être un goulot d'étranglement si le nombre de variables du calcul parallèle devient trop élevé.

Tous les paramètres ci-dessus jouent un rôle clé dans l'évaluation des performances d'un GPU ; cependant, le facteur économique et environnemental est crucial dans le choix de la carte. Ainsi, le rapport TFLOPS/prix est le premier facteur pris en compte dans le choix d'une unité de calcul GPU. De plus, les GPU consomment énormément de puissance électrique pour fonctionner (de 100 à 500 watts) et croissent davantage au fil des générations. Une grande partie de la puissance introduite se trouve dissipée sous forme de chaleur. Mais, pour bien fonctionner, les centres de calcul doivent maintenir la température des GPU à une température convenable (entre 50 et 80 °C), alors que leur fonctionnement est exothermique⁴.

3.3 CUDA

CUDA signifie "Compute Unified Device Architecture". C'est une plateforme de calcul parallèle développée par NVIDIA qui permet d'utiliser les GPU NVIDIA pour des calculs généraux, et pas seulement pour le rendu graphique.

CUDA utilise une organisation appelée SIMT (Single Instruction, Multiple Threads), une extension de SIMD dans la taxonomie de Flynn. Un thread est une instruction de calcul à exécuter. Une partie des threads est regroupée en blocs. Les threads d'un même bloc peuvent collaborer et partager certaines données via la mémoire partagée du GPU. Les blocs sont regroupés dans le grid qui constitue l'ensemble du noyau (kernel) CUDA. Tous les threads exécutent généralement le même noyau, mais chaque thread travaille sur des données différentes. Pour l'addition de deux vecteurs, cela prend la forme suivante

```
1 __global__ void addition(float *A, float *B, float *C)
```

⁴De plus en plus de supercalculateurs (tel Jean Zay) utilisent un système de refroidissement par eau chaude à cœur qui récupère une grande partie des calories émises pour chauffer les réseaux d'urbanisme.

```
2 {  
3     int i = threadIdx.x;  
4     C[i] = A[i] + B[i];  
5 }
```